

A Highly Efficient Numerical Method to Solve Non-linear Functions Using Barycentric Interpolation

Enrique Aguilar-Méndez and Roxana M. del Castillo

Departamento de Física, Facultad de Ciencias
Universidad Nacional Autónoma de México
Circuito interior s/n, Ciudad Universitaria
Mexico City 04510, Mexico

This article is distributed under the Creative Commons by-nc-nd Attribution License.
Copyright © 2021 Hikari Ltd.

Abstract

We developed a novel numerical method for finding the root of non-linear functions. The algorithm developed uses inverse barycentric interpolation to catch the root into a given interval. The main advantage of using this new method is to guarantee fast convergence. In addition, the barycentric interpolation is constructed to eliminate Runge noise, which appears whenever homogeneous nodes are used, complicating the process of finding a solution. First, we enclose the root between two points obtained with quadratic barycentric interpolation. Then, we create a third point with a cubic barycentric interpolation such that the root will be trapped in some interval of these three points. This selection of points ensures a rapid convergence of the method. The new algorithm makes use of a parameter k which acts as a selector of precision and speed. We obtained excellent results in terms of the solution of multiple roots, computational time, convergence, and error.

Keywords: Numerical Method, Finding Method, New Algorithm, Root finding, Nonlinear functions Barycentric formula, Extreme precision

1 Introduction

The numerical solution of nonlinear equations is one of the most critical problems of modern numerical analysis, computational physics, and scientific computation [1, 2, 3, 4, 5, 6, 7]. Such is the importance of having efficient and

precise algorithms that the development of new methods continues to be a hot topic in scientific computation. In the literature, there are many algorithms to solve these types of equations, and each one of them has a good list of pros and cons associated with them, which must be evaluated before using the algorithm in question. The user has a wide range of possibilities to choose from; however, not all the available algorithms work well or guarantee convergence. This situation obligates the user to understand the nature of the problem very well; for example, it has to be known if the function is differentiable, no longer continuous, or if the function has multiple roots [8, 9]. The need for better algorithms has been so important that hybrid methods have been developed to encompass different types of functions and improve their efficiency and convergence. The most famous example is the Brent-Dekker method, published in 1973 [10], which combines bisection, secant, and inverse quadratic interpolation, making this algorithm robust and widely popular. The main Achilles' heel of the Brent method is to work with multiple roots. In 1996, Chandrupatla [11], in 1996, solved the multiple roots of the Brent method and improved the interpolation by proposing a hybrid algorithm that continued with the idea of using quadratic interpolation and the bisection method. Specifically, the Chandrupatla algorithm uses three different points in each step of the iteration and approximates them with a second-order interpolated polynomial, where, the selection of the interval is based on the bisection method. The main advantage this method relies on its accuracy for higher multiples roots, like x^{10} or x^{13} . Then, in 2010, Gottlieb and Thompson [12] created another bisected direct quadratic regulate falsi (BDQRF) algorithm using quadratic interpolation demonstrating to have rapid convergence of root finding. In the algorithms described above, quadratic interpolation is used without solving the Runge problem; thus, one can then expect that there will be functions for which interpolation with homogeneous nodes will induce oscillations and convergence will not be guaranteed [13]. We propose a new method that uses barycentric interpolation with inhomogeneous nodes to avoid the Runge phenomenon by decreasing the induced oscillations and therefore, improving accuracy and convergence. We will use four points to evaluate and then, these points will be approximated by an interpolated function based on barycentric interpolation. The algorithm was tested for some complicated functions and showed substantial improvement in the solution of multiple roots, reduction of computational time, rate of convergence, and error reduction.

2 Derivation

The core idea of the method is to trap the root between two points, which, are generated with the quadratic, inverse barycentric interpolation with three inhomogeneous nodes in order to approach the root. Then, a third point is

generated using cubic inverse barycentric interpolation to approach the root. This third point takes the advantage of the recursion nature of the barycentric interpolation such that the weights are obtained from the previous weights used by the quadratic barycentric interpolation. In the following, a schematic presentation of the mathematical foundation of our method is presented.

2.1 Barycentric interpolation of the inverse function

It is well known that for every continuous function $f(x) \in [a, b]$ there is a set of polynomials $P_n(x)$ with $n = 1, 2, \dots$ (where P_n is a polynomial of degree n), with convergence. This statement is the Weierstrass approximation theorem [14]:

$$\lim_{n \rightarrow \infty} [\max_{a \leq x \leq b} |f(x) - P_n(x)|] = 0$$

A natural supposition of the Weierstrass approximation theorem is that, using more points will lead to a more accurate reconstruction of the function. However, the theorem does not guarantee to possess the property of uniform convergence. The $P_n(x)$ constructed in the conventional way may deviate from the original function $f(x)$ as n increases, giving rise to an oscillating pattern that magnifies the error of the interpolation. This effect is the so-called Runge phenomenon [15, 16].

The barycentric interpolation is a correction to the Lagrange interpolation in respect to controlling the Runge phenomenon. The latter appears as a consequence of two main properties:

- The magnitude of the n -th order derivatives overgrows when n increases.
- The equidistant nodes lead to a Lebesgue constant that increases quickly as a function of the number of nodes n .

The Lebesgue constant for a homogeneous grid in the Lagrange interpolation $L(x) = \sum_{i=0}^n \frac{x-x_i}{x_j-x_i}$ with the Lebesgue function $\lambda_n(x) = \sum_{j=0}^n |L_j(x)|$ is [17, 18]:

$$\Lambda_n(T) = \max_{x \in [a, b]} \lambda_n(x).$$

Now, the correction made by the barycentric interpolation is a manipulation of Lagrange's interpolation formula that induces recursion and allows to introduce inhomogeneous nodes at Lagrange's interpolation [13, 19]. For a given real-valued function $f(x)$, the barycentric interpolation is given by

$$P(x) = \frac{\sum_{j=0}^n \frac{w_j f_j}{x_j - x}}{\sum_{j=0}^n \frac{w_j}{x_j - x}}, \quad (1)$$

where x_j are the nodes, $w_j = \prod_{i \neq j} \frac{1}{x_i - x_j}$ are the weights introduced to improve the interpolation, and the new Lebesgue constant is [13, 20]

$$\Lambda \geq \frac{1}{2n^2} \frac{\max_{0 \leq j \leq n} |w_j|}{\min_{0 \leq j \leq n} |w_j|}.$$

The inverse barycentric interpolation consisted in obtaining the $x_{intercept}$ which represented the x value that intercepted the x -axis, i.e., $f(x) = 0$. The $x_{intercept}$ is given by:

$$x_{intercept} = \frac{\sum_{j=0}^n \frac{w_j x_j}{f_j}}{\sum_{j=0}^n \frac{w_j}{f_j}} \quad (2)$$

Here $w_j = \prod_{i \neq j} \frac{1}{f_i - f_j}$. Equation (2) is the formula used in this new algorithm.

2.2 Description of the algorithm

Our algorithm begins with the function to evaluate $f(x)$, a test interval $[x_0, x_1]$ enclosing the root, and the precision or tolerance desired for the root. To check for the existence of the root in the interval, we apply the intermediate value theorem; which signals the existence of a root in the interval $[x_0, x_1]$ if the function evaluated at these points presents a change of sign, i.e., $sign(f(x_0)) \neq sign(f(x_1))$ or, equivalently, $f(x_0)f(x_1) < 0$.

We present the main steps of the algorithm to find roots of nonlinear functions. The pseudo-code is presented in the appendix section and the code in Python code is available at <https://github.com/EnriqueAguilarM/A-highly-efficient-numerical-method-to-solve-non-linear-functions-using-barycentric-interpolation>.

- **Step 1:** The root existence is verified with $sign(f(x_0)) \neq sign(f(x_1))$. If the condition is true, we create a new point $x_2 = x_0 + k(x_1 - x_0)$ in the interval, which uses an arbitrary parameter k between 0 and 0.5, which can be used as the precision parameter of the algorithm (we will address this in detail in the next section).
- **Step 2:** The intervals $[x_0, x_2]$ and $[x_2, x_1]$ are evaluated. If the root falls in $[x_0, x_2]$, we return at the starting point of the cycle to try a smaller new interval. For the case in which the root is in $[x_2, x_1]$, the next point of the sequence is generated with $x_3 = x_1 + k(x_0 - x_1)$.

- **Step 3:** There are two intervals to evaluate, namely, $[x_2, x_3]$ and $[x_3, x_1]$. If $\text{sign}(f(x_3)) \neq \text{sign}(f(x_1))$ we return to the beginning of the cycle to test the new interval $[x_3, x_1]$. Otherwise, the interval we take is $[x_2, x_3]$ and we calculate two new points x_4 and x_5 . These points are constructed through a quadratic inverse barycentric interpolation with the sets of points $\{x_0, x_2, x_3\}$ and $\{x_2, x_3, x_1\}$, respectively. The equations of these points are:

$$\begin{aligned} x_4 &= \frac{w_0x_0 + w_2x_2 + w_3x_3}{w_0 + w_2 + w_3}, \\ x_5 &= \frac{w_1x_1 + w_2x_2 + w_3x_3}{w_0 + w_2 + w_3}, \end{aligned}$$

with the weights:

$$w_0 = \frac{1}{f_0(f_0 - f_2)(f_0 - f_3)}, \quad w_2 = \frac{1}{f_2(f_2 - f_0)(f_2 - f_3)}, \quad w_3 = \frac{1}{f_3(f_3 - f_0)(f_3 - f_2)}.$$

$$w_1 = \frac{1}{f_1(f_1 - f_2)(f_1 - f_3)}, \quad w_{22} = \frac{1}{f_2(f_2 - f_1)(f_2 - f_3)}, \quad w_{23} = \frac{1}{f_3(f_3 - f_1)(f_3 - f_2)}.$$

- **Step 4:** We evaluate the existence of the points x_4 and x_5 in the interval $[x_2, x_3]$. The following cases have to be considered:
 - If x_4 is not in the interval but x_5 is, we choose a new interval with x_5 and x_2 or x_3 , such that the selected interval has the root.
 - If x_4 is in the interval and x_5 is not, we take the new interval with x_4 and x_2 or x_3 , such that the selected interval has the root.
 - If none of the points is in the interval, we continue with the previous interval $[x_2, x_3]$.
 - If both points are in the interval, the new interval is $[\min(x_4, x_5), \max(x_4, x_5)]$.
- **Step 5:** With the interval obtained in Step 4 it is possible to obtain the next point x_6 , which is the approximation of the root. The point x_6 is obtained with a cubic inverse barycentric interpolation and the inhomogeneous nodes x_0, x_1, x_2 , and x_3 , using the iterations and the weights obtained on Step 3.

$$x_6 = \frac{w_0x_0 + w_1x_1 + w_2x_2 + w_3x_3}{w_0 + w_1 + w_2 + w_3} \quad (3)$$

with the weights

$$w_0 = \frac{w_0}{(f_0 - f_1)}, \quad w_1 = \frac{w_1}{(f_1 - f_0)}, \quad w_2 = \frac{w_2}{(f_2 - f_1)}, \quad w_3 = \frac{w_3}{(f_3 - f_1)}.$$

We evaluate the existence of x_6 in the interval. Again, there are two cases to analyze:

- If $x_6 \in [x_2, x_3]$ we evaluate on which side is the root located. If $\text{sign}(f(x_6)) \neq \text{sign}(f(x_4))$, then the new interval is $[\min(x_4, x_6), \max(x_4, x_6)]$. Otherwise, the new interval is given by $[\min(x_5, x_6), \max(x_5, x_6)]$.
- If x_6 is not in the interval $[x_2, x_3]$, we continue with the previous interval $[\min(x_4, x_5), \max(x_4, x_5)]$.

With these steps, we guarantee that the root is found in side of this interval. We conclude that the root is around the half-point of the interval, $x = \frac{(x_0+x_1)}{2}$.

The process is iterated over as many times as necessary to meet the provided tolerances.

3 Results

3.1 Numerical Testing

The newly developed algorithm was tested against Brents's method (BR), Chandrupatla's method (CH) and the "Bisected Direct Quadratic Regula Falsi" method (BDQRF). All the functions given in [12] were tested as well as some given in Ref. [11]. In all cases, $\text{abs}(x_0 - x_1) < \text{tolerance}$, where $[x_0, x_1]$ is the interval where the root is bracketed, was used as convergence criterion. The tolerance was set to 10^{-6} and $k = \frac{1}{6}$.

As can be seen from Table 1, in most cases, the new method improves or at least equals the other methods in terms of the number of iterations required to obtained the root within the margin of tolerance. In the literature, there are other methods that use barycentric interpolation to find the roots of nonlinear functions, such as those using Chebyshev or Hermite nodes [21, 22, 23, 24]. However, we propose to use the interval divisions to accelerate convergence and keep the stability of the barycentric interpolation. So, the nodes will depend on both the function to evaluate and the test interval whilst convergence is guaranteed as a consequence of the interval splitting.

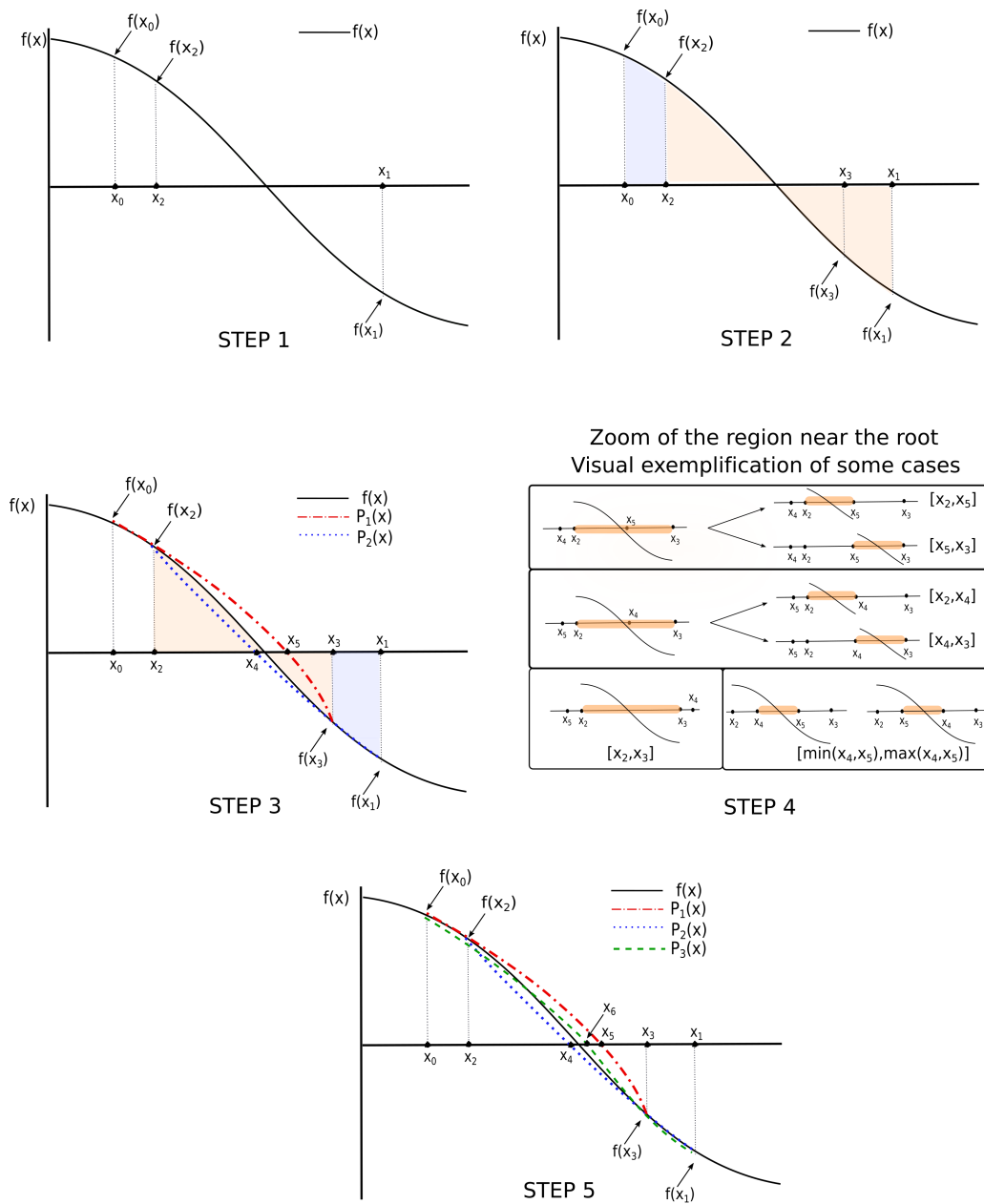


Figure 1: Schematic representation of the main steps of the algorithm.

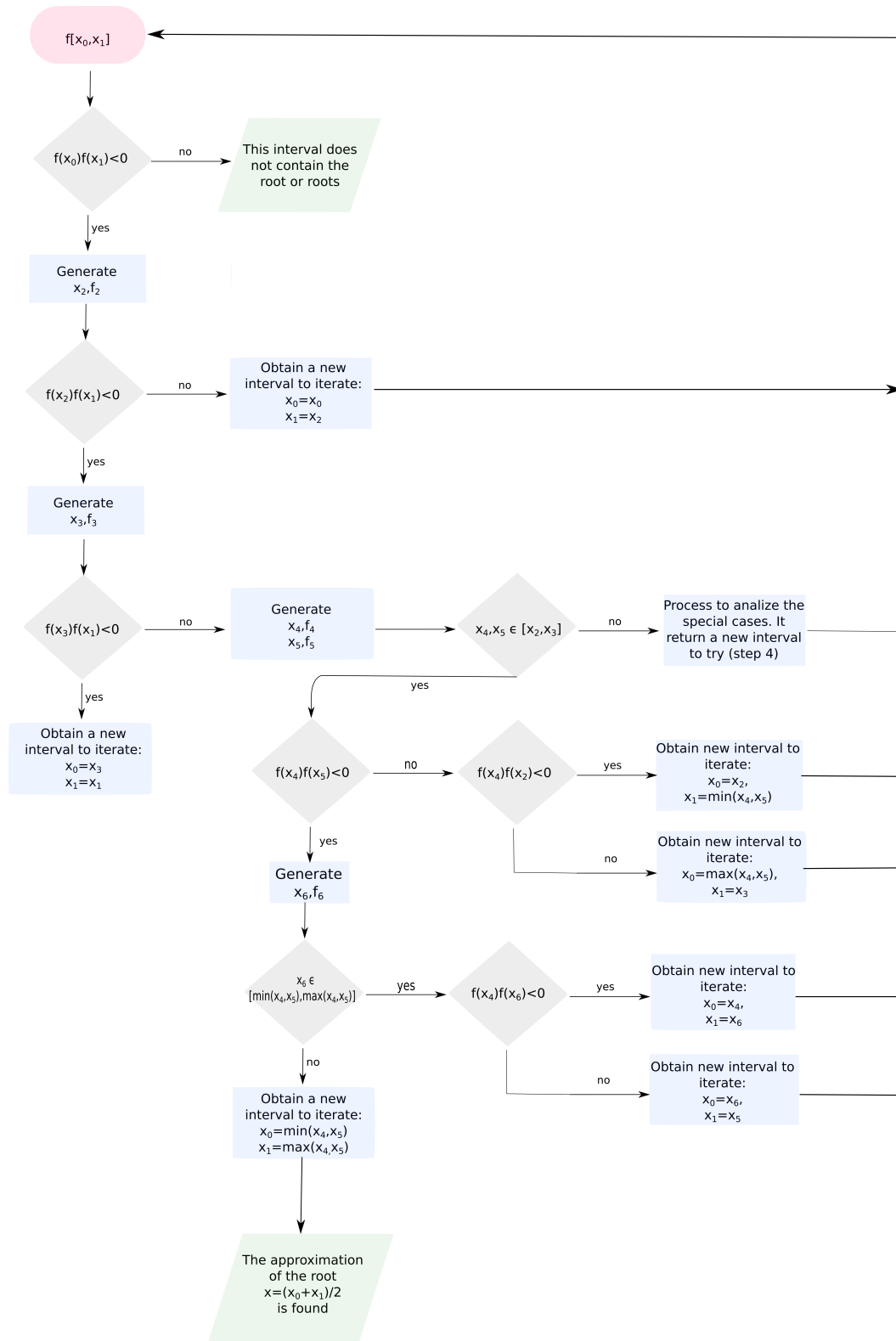


Figure 2: Flux Diagram of the Algorithm.

Function	Interval	Method				x
		BR	CH	BDQRF	NEW	
$3 \sin(x) - 2 = 0$	[0.1]	10	6	6	2	0.729727
$xe^x - 1 = 0$	[-1,1]	14	6	5	4	0.567143
$11x^{11} - 1 = 0$	[0.1,0.9]	21	8	5	3	0.804133
$e^{x^2+7x-30} - 1 = 0$	[2.8,3.1]	10	7	5	3	2.999999
$\frac{1}{x} - \sin(x) + 1 = 0$	[-1.3,-0.5]	13	6	4	3	-0.629446
$\frac{1}{x} - 1 = 0$	[0.5,1.5]	12	6	4	4	0.999999
$x^3 - 2x - 5 = 0$	[2.5,2.0]	12	5	4	4	2.094551
$(x - 3)^3 = 0$	[2,4.1]	27	25	19	11	2.999999
$6(x - 2)^5 = 0$	[1,3.1]	29	26	21	17	2.000000
$x^9 = 0$	[-1.2,4]	29	23	23	23	0.000000

Table 1: Numerical results of the new method. It is shown a comparison between the new method and other methods given in literature [10, 11, 12]

3.2 The parameter k

Let f be a function defined on the interval $\Delta x = [x_0, x_1]$. In Step 1 and Step 2 of the previous section, the interval was split using $x_2 = x_0 + k(x_1 - x_0)$ and $x_3 = x_2 + k(x_1 - x_2)$ with $k \in (0, 0.5]$. In Step 1, the interval is split in two new intervals of length $k\Delta x$ and $(1 - k)\Delta x$. As seen in the description of the algorithm, the selection of the interval to continue the iteration is $(1 - k)\Delta x$. Then, in Step 2, we repeated the subdivision and selection of the interval. Once again, the interval $(1 - k)\Delta x$ is divided in two intervals of length $k\Delta x$ and $((1 - k) - k)\Delta x = (1 - 2k)\Delta x$, respectively. We select the interval of length $(1 - 2k)\Delta x$. So, in the first step, the interval is reduced to:

$$\Delta x^{(1)} = (1 - 2k)\Delta x$$

In the second step, the new interval is proportional to the original interval as:

$$\Delta x^{(2)} = (1 - 2k)\Delta x^{(1)} = (1 - 2k)^2\Delta x$$

and so on until reaching the n -th step

$$\Delta x^{(n)} = (1 - 2k)\Delta x^{(n-1)} = (1 - 2k)^n\Delta x = ((1 - 2k)^n\Delta x$$

With these two cuts in each step, we make sure that we get a smaller interval after each iteration. By varying k , we can choose how close will x_2 and x_3 be to the interval boundaries. The selection of the variable k determines whether x_4 and x_5 are reasonable estimates for the roots. We proposed different choices of k for various functions. We found that $k = \frac{1}{6}$ does well for faster convergence. For complicated functions in which there is a division of very small numbers,

we recommend setting k to $\frac{1}{2}$, which returns the bisection method in each step, allowing greater precision to be handled, although it takes longer. To control this parameter, we introduced a variable called “bisection threshold” which indicated the precision to applied k as $\frac{1}{2}$.

3.3 Error and Numerical Stability

To estimate the rate of convergence we used the absolute *error* defined as $|x_{real} - x_{approx}|$. The *error* was plotted versus the amount of iterations needed to reach a tolerance of 10^{-10} in order to obtain the rate of convergence for the new method. The test functions used were $(x - 3)^6$ and $6(x - 2)^5$. In Fig 3 the error versus the number of iterations is shown for both test functions and for three different methods. A faster convergence is seen when compared to the other methods, which are quite fast in themselves. The novel method that we propose overcomes in convergence and solves multiple-root functions.

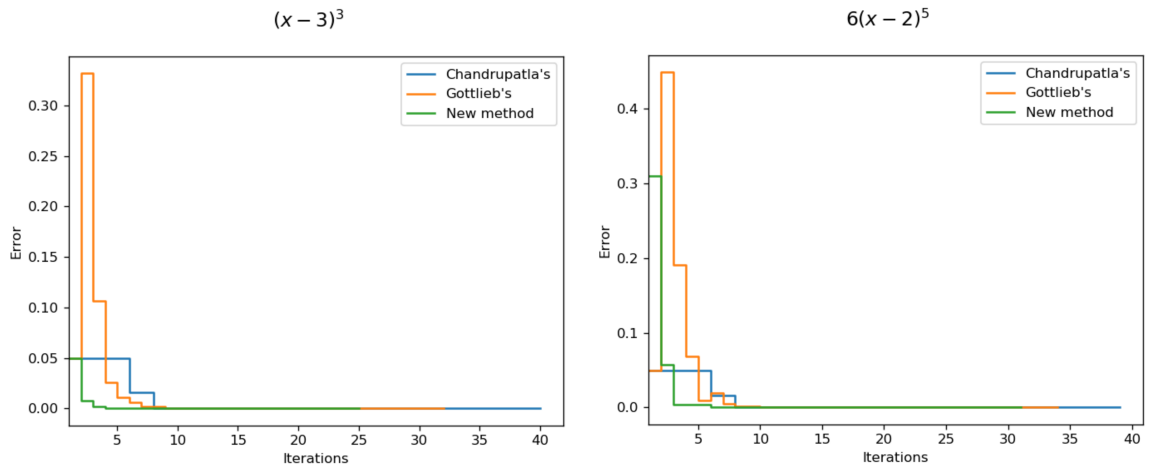


Figure 3: Rate of convergence of two test functions, $(x - 3)^6$ and $6(x - 2)^5$, respectively.

Frequently, polynomial interpolation introduces errors because of computer arithmetic, a situation that can represent a problem in the root-finding calculations. However, the barycentric formula will be stable due the weight constructions [25]. Notice that may be argued that $w_k/(x - x_k)$ would be large enough to induce numeric error but, according to Henrici [26], this cannot happen since the inaccuracies that appear in the numerator and denominator cancel out.

It is possible, however, to have division by zero errors in the calculation of the weights and the root estimate with the barycentric interpolation formula. Verifying that the function is different from zero when evaluated in the extremes of the interval as well as in the generated points is necessary. It may

also be possible that the function evaluated in these points is not exactly zero but the product of these values is so small that it can give rise to a division-by-zero error is raised. This last problem is particularly troublesome when dealing with multiple roots because in these cases, the values of the function decrease very rapidly as the precision increases. To avoid this situation, we set a “bisection threshold”. If the function evaluated in the extremes of the interval is smaller than this threshold k is set to $\frac{1}{2}$ and the algorithm is thus converted into bisection. For the tests above, this threshold was set to 10^{-17} , but it can be modified by the user can be modified.

4 Conclusions

A new method has been proposed to find the solution of nonlinear equations. This method is based on catching the root or roots between two points, which are obtained with the inverse barycentric interpolation. These two points are obtained from the previous three points by splitting the interval and work as inhomogeneous nodes for the barycentric interpolation. We compared the new method with three well-known methods recently published and we observed very good results.

From the numerical convergence and stability analysis, it has been shown that the method is highly efficient. The method is constituted such that the user can select the value of the parameter k useful for faster convergence and extreme precision. For fast convergence, we recommend to set k as $1/6$, while for functions with small numbers divisions, we recommend to set k as $\frac{1}{2}$, which returns the bisection method in each step, allowing greater precision to be handled.

The method has been devised for complicated functions for it reduces numerical instabilities induced by conventional interpolations, like Lagrange or Newton. The reduction of instabilities comes from the fact that we are using the barycentric approach, where the weights introduced work to eliminate said instabilities due to their construction. For very complicated functions, the Runge instabilities introduced by the construction of the nodes in the interpolation were found to decrease.

Acknowledgements. This research was funded by Direccin General de Asuntos de Personal Acadmico (DGAPA), Universidad Nacional Autnoma de Mxico (UNAM) with the project IN120120. Also, this work was carried out using Miztli HP Cluster 3000 supercomputer, provided by Direccin General de Cmputo y Tecnologas de Informacin y Comunicacin (DGTIC), Universidad Nacional Autnoma de Mxico (UNAM) with the projects LANCAD-UNAM-DGTIC-385.

5 Appendix

The Python code is available at <https://github.com/EnriqueAguilarM/A-highly-efficient-numerical-method-to-solve-non-linear-functions-using-barycentric-interpolation>. Here, we present the pseudo-code:

```

def new_method (f,a,b,tol,k=1/6,bisection_threshold=10**(-17)):
     $x_0, x_1 = a, b$ 
    while  $\text{abs}(x_0 - x_1) > \text{tol}$ :
         $f_0, f_1 = f(x_0), f(x_1)$ 
        if  $f_0 == 0$  :
             $x_0, x_1 = x_0, x_0$ 
            break
        if  $\text{abs}(f_0) < \text{bisection\_threshold}$  or  $\text{abs}(f_1) < \text{bisection\_threshold}$ :
             $k=0.5$ 
        if  $f_1 == 0$  :
             $x_0, x_1 = x_1, x_1$ 
            break
         $x_2 = x_0 + k \cdot (x_1 - x_0); f_2 = f(x_2)$ 
        if  $f_2 == 0$  :
             $x_0, x_1 = x_2, x_2$ 
            break
        if  $f_0 \cdot f_2 < 0$ :
             $x_0, x_1 = x_0, x_2$ 
        elif  $f_0 == f_2$ :
             $x_0, x_1 = x_2, x_1$ 
        else:
             $x_3 = x_1 + k \cdot (x_0 - x_1)$ 
             $f_3 = f(x_3)$ 
            if  $f_3 == 0$ :
                 $x_0, x_1 = x_3, x_3$ 
                break
            if  $f_3 \cdot f_1 < 0$ :
                 $x_0, x_1 = x_3, x_1$ 
            elif  $f_3 == f_1$ :
                 $x_0, x_1 = x_2, x_3$ 
            else:
                 $w_0 = 1/(f_0 \cdot (f_0 - f_2) \cdot (f_0 - f_3))$ 
                 $w_2 = 1/(f_2 \cdot (f_2 - f_0) \cdot (f_2 - f_3))$ 
                 $w_3 = 1/(f_3 \cdot (f_3 - f_0) \cdot (f_3 - f_2))$ 
                 $x_4 = (w_0 \cdot x_0 + w_2 \cdot x_2 + w_3 \cdot x_3)/(w_0 + w_3 + w_2)$ 
                 $f_4 = f(x_4)$ 
                 $w_1 = 1/(f_1 \cdot (f_1 - f_2) \cdot (f_1 - f_3))$ 

```

```

 $w_{22} = 1/(f_2 \cdot (f_2 - f_1) \cdot (f_2 - f_3))$ 
 $w_{32} = 1/(f_3 \cdot (f_3 - f_1) \cdot (f_3 - f_2))$ 
 $x_5 = (w_1 \cdot x_1 + w_{22} \cdot x_2 + w_{32} \cdot x_3)/(w_1 + w_{32} + w_{22})$ 
 $f_5 = f(x_5)$ 
if  $x_4 \leq x_2$  or  $x_4 \geq x_3$ :
    if  $x_5 \leq x_2$  or  $x_5 \geq x_3$ :
         $x_0, x_1 = x_2, x_3$ 
    else:
        if  $f_5 \cdot f_2 < 0$ :
             $x_0, x_1 = x_2, x_5$ 
        else:
             $x_0, x_1 = x_5, x_3$ 
else:
    if  $x_5 \leq x_2$  or  $x_5 \geq x_3$ :
        if  $f_4 \cdot f_2 < 0$ :
             $x_0, x_1 = x_2, x_4$ 
        else:
             $x_0, x_1 = x_4, x_3$ 
    else:
        if  $f_5 \cdot f_4 > 0$ :
            if  $f_4 \cdot f_2 < 0$ :
                 $x_0, x_1 = x_2, \min(x_4, x_5)$ 
            else:
                 $x_0, x_1 = \max(x_4, x_5), x_3$ 
        else:
             $w_0 = w_0/(f_0 - f_1)$ 
             $w_1 = w_1/(f_1 - f_0)$ 
             $w_2 = w_2/(f_2 - f_1)$ 
             $w_3 = w_3/(f_3 - f_1)$ 
             $x_6 = \frac{w_0 \cdot x_0 + w_1 \cdot x_1 + w_2 \cdot x_2 + w_3 \cdot x_3}{w_0 + w_1 + w_3 + w_2}$ 
             $f_6 = f(x_6)$ 
            if  $x_6 \leq x_2$  or  $x_6 \geq x_3$ :
                 $x_0, x_1 = \min(x_4, x_5), \max(x_4, x_5)$ 
            else:
                if  $f_6 \cdot f_4 < 0$ :
                     $x_0, x_1 = \min(x_4, x_6), \max(x_4, x_6)$ 
                else:
                     $x_0, x_1 = \min(x_5, x_6), \max(x_5, x_6)$ 

return  $(x_0 + x_1)/2$ 

```

References

- [1] E. Hansen, M. Patrick, A family of root finding methods, *Numerische Mathematik*, **27** (1976), 257-269. <https://doi.org/10.1007/BF01396176>
- [2] P. D. Proinov, Relationships between different types of initial conditions for simultaneous root finding methods, *Applied Mathematics Letters*, **52** (2016), 102-111. <https://doi.org/10.1016/j.aml.2015.08.016>
- [3] M. Petkovic, *Point Estimation of Root Finding Methods*, Lecture Notes in Mathematics, Springer-Verlag Berlin Heidelberg, Tiergartenstrasse 17, 69121 Heidelberg, Alemania, 2008.
- [4] A. Sergio, B. Sonia, S. Plaza, Review of some iterative root-finding methods from a dynamical point of view, *SCIENTIA, Series A: Mathematical Sciences*, **10** (2004), 3-35.
- [5] C. Chun, Y. Ham, Newton-like iteration methods for solving non-linear equations, *Communications in Numerical Methods in Engineering*, **22** (2006), 475-487. <https://doi.org/10.1002/cnm.832>
- [6] F. Zafar, A. Cordero, J. R. Torregrosa, A. Rafi, A class of four parametric with- and without-memory root finding methods, *Computational and Mathematical Methods*, **1** (2019), e1024. <https://doi.org/10.1002/cmm4.1024>
- [7] C. L. Howk, J. L. Hueso, E. Martinez, C. Teruel, A class of efficient high-order iterative methods with memory for nonlinear equations and their dynamics, *Mathematical Methods in the Applied Sciences*, **41** (2018), 7263-7282. <https://doi.org/10.1002/mma.4821>
- [8] M. Reuter, T. S. Mikkelsen, E. C. Sherbrooke, T. Maekawa, N. M. Patrikalakis, Solving nonlinear polynomial systems in the barycentric bernstein basis, *The Visual Computer*, **24** (2008), 187-200. <https://doi.org/10.1007/s00371-007-0184-x>
- [9] T. Ojika, S. Watanabe, T. Mitsui, Deflation algorithm for the multiple roots of a system of nonlinear equations, *Journal of Mathematical Analysis and Applications*, **96**, (1983), 463-479. [https://doi.org/10.1016/0022-247X\(83\)90055-0](https://doi.org/10.1016/0022-247X(83)90055-0)
- [10] R. P. Brent, *Algorithms for minimization without derivatives*, Prentice-Hall series in automatic computation, Prentice-Hall, Englewood Cliffs, NJ, 1973. URL: <http://cds.cern.ch/record/113464>

- [11] T. R. Chandrupatla, A new hybrid quadratic/bisection algorithm for finding the zero of a nonlinear function without using derivatives, *Advances in Engineering Software*, **28** (1997), 145-149. URL: <http://www.sciencedirect.com/science/article/pii/S0965997896000518>, [https://doi.org/10.1016/S0965-9978\(96\)00051-8](https://doi.org/10.1016/S0965-9978(96)00051-8)
- [12] R. G. Gottlieb, B. F. Thompson, Bisected direct quadratic regula falsi, *Applied Mathematical Sciences*, **4** (2010), 709-718. URL: http://www.m-hikari.com/ams/ams-2010/ams-13-16-2010/gottliebAMS13-16-2010.pdf?ev=pub_ext_prw_xdl
- [13] J. P. Berrut, L. N. Trefethen, Barycentric lagrange interpolation, *SIAM Review*, **46** (2004), 1-17. <https://doi.org/10.1137/S0036144502417715>
- [14] R. Vyborn, The Weierstrass theorem on polynomial approximation, *Mathematica Bohemica*, **130** (2005), 161-166. <https://doi.org/10.21136/MB.2005.1341325>
- [15] B. Fornberg, J. Zuev, The runge phenomenon and spatially variable shape parameters in rbf interpolation, *Computers Mathematics with Applications*, **54** (2007), 379-398. <https://doi.org/10.1016/j.camwa.2007.01.028>
- [16] J. F. Epperson, On the runge example, *The American Mathematical Monthly*, **94** (1987), 329-341. <https://doi.org/10.1080/00029890.1987.12000642>
- [17] L. Brutman, On the lebesgue function for polynomial interpolation, *SIAM Journal on Numerical Analysis*, **15** (1978), 694-704. <https://doi.org/10.1137/0715046>
- [18] B. A. Ibrahimoglu, Lebesgue functions and lebesgue constants in polynomial interpolation, *Journal of Inequalities and Applications*, **2016** (2016), 93. <https://doi.org/10.1186/s13660-016-1030-3>
- [19] M. Polezzi, A. Sri Ranga, On the denominator values and barycentric weights of rational interpolants, *Journal of Computational and Applied Mathematics*, **200**, (2007), 576-590. <https://doi.org/10.1016/j.cam.2006.01.013>
- [20] L. Bos, S. De Marchi, K. Hormann, G. Klein, On the Lebesgue constant of barycentric rational interpolation at equidistant nodes, *Numerische Mathematik*, **121** (2012), 461-471. <https://doi.org/10.1007/s00211-011-0442-8>
- [21] R. Corless, A. Shakoory, D. Aruliah, L. Gonzalez-Vega, Barycentric hermite interpolants for event location in initial-value problems, *European Society of Computational Methods in Sciences and Engineering (ESCMSE)*,

- Journal of Numerical Analysis, Industrial and Applied Mathematics*, **3** (2008), 1-16.
- [22] M. S. Petkovic, L. Rancic, L. D. Petkovic, S. Ilic, Chebyshev-like root-finding methods with accelerated convergence, *Numerical Linear Algebra with Applications*, **16** (2009), 971-994. <https://doi.org/10.1002/nla.661>
 - [23] P. Lawrence, R. Corless, Stability of root finding for barycentric Lagrange interpolants, *Numerical Algorithms*, **65** (2014), 447-464. <https://doi.org/10.1007/s11075-013-9770-3>
 - [24] K. Gdawiec, Fractal patterns from the dynamics of combined polynomial root finding methods, *Nonlinear Dynamics*, **90** (2017), 2457-2479. <https://doi.org/10.1007/s11071-017-3813-6>
 - [25] N. J. Higham, The numerical stability of barycentric Lagrange interpolation, *IMA Journal of Numerical Analysis*, **24** (2004), 547-556. <https://doi.org/10.1093/imanum/24.4.547>
 - [26] P. Henrici, Barycentric formulas for interpolating trigonometric polynomials and their conjugates, *Numerische Mathematik*, **33** (1979), 225-234. <https://doi.org/10.1007/BF01399556>

Received: April 29, 2021; Published: June 4, 2021